ETAS ISOLAR-A 12.7.0

API DOCUMENTATION(Scripting)

www.etas.com

Copyright

The data in this document may not be altered or amended without special notification from ETA S GmbH. ETAS GmbH undertakes no further obligation in relation to this document. The software described in it can only be used if the customer is in possession of a general license agreement or single license. Using and copying is only allowed in concurrence with the specifications stipulated in the contract.

Under no circumstances may any part of this document be copied, reproduced, transmitted, stored in a retrieval system or translated into another language without the express written permission of ETAS GmbH.

© Copyright 2025 ETAS GmbH, Stuttgart

The names and designations used in this document are trademarks or brands belonging to the respective owners.

ETAS ISOLAR-A 12.7.0 – API DOCUMENTATION(Scripting) R01 EN – 01.2025

Contents

1 Introduction 4

1.1 Definitions, Acronyms and Abbreviations 4

1.2 Conventions 4

1.3 Disclaimer 4

1.4 About You 5

2 API Documentation 6

2.1 Java Scripting API‘s 6
2.1.1 Categories of API’s 6
2.1.2 ARTOP Utils 6
2.1.3 AR Software 12
2.1.4 AR System 19

Description : 19
2.1.5 ISOLAR Utils 20

2.2 Java Action APIs 26
2.2.1 Categories of API’s 26
2.2.2 ASWLibrary Utils 27
2.2.3 JavaActionUtility 38
2.2.4 ASWModel 40
2.2.5 Artop Utils 45

3 Example Projects 73

3.1 Java Action Example Project 73

3.2 Scripting Example Projects 73

4 Contact Information 75

1 Introduction

ISOLAR-A is the ETAS AUTOSAR Authoring Tool that assists users in designing application software to AUTOSAR standards.

This document provides information for using the ISOLAR-A Scripting Environment. The scripting environment can be used to develop, manage and execute scripts within ISOLAR-A for a variety of use-cases.

Some of the use cases where Scripting can be useful are – AUTOSAR Model Updation, Automation of feature invocation, Resource manipulation, Search and Reporting.

1.1 Definitions, Acronyms and Abbreviations

AUTOSAR: AUTomotive Open System ARchitecture - standardized software architecture targeted at automotive applications software aiming to foster the re-use of application software over multiple vehicle platforms. For further details, please see http://www.autosar.org

1.2 Conventions

The following typographical conventions are used in this document:

OCI_CANTxMessage msg0 =	Code snippets are presented on a gray background and in the Courier font. Meaning and usage of each command are explained by means of comments. The comments are enclosed by the usual syntax for comments.
Choose File → Open.	Menu commands are shown in boldface.
Click OK.	Buttons are shown in boldface.
Press <ENTER>.	Keyboard commands are shown in angled brackets.
The "Open File" dialog box is displayed.	Names of program windows, dialog boxes, fields, etc. are shown in quotation marks.
Select the file setup.exe	Text in drop-down lists on the screen, program code, as well as path- and file names are shown in the Courier font.
A *distribution* is always a one-dimensional table of sample points.	General emphasis and new terms are set in italics.

1.3 Disclaimer

ETAS GmbH cannot be made liable for any damage which is caused by incorrect use and not adhering to the safety instructions. Use of scripts, regardless of those shipping with the product and of those developed or otherwise modified by whoever, is in the sole responsibility of users. You are solely responsible for determining the appropriateness of using the scripts and assume any risks associated with your use of the scripts. In no event and under no legal theory, whether in tort (including negligence), contract, or otherwise, unless required by applicable law (such as deliberate and grossly negligent acts) or agreed to in writing, shall ETAS be liable to you for damages, including any direct, indirect, special, incidental, or consequential damages of any character arising as a result out of the use or

inability to use the scripts (including but not limited to damages for loss of goodwill, work stoppage, computer failure or malfunction, or any and all other commercial damages or losses), even if ETAS has been advised of the possibility of such damages.

1.4 About You

You are a trained Systems Integrator with experience is scripting languages like Javascript and/or Python. You are familiar with AUTOSAR methodology and its meta model. You are also familiar with ARTOP, Eclipse framework and its resource management. This document contains usage of APIs that work with the Eclipse framework and the Eclipse Modeling Framework (EMF) realization of the AUTOSAR meta-model.

You should also be familiar with common use of the Microsoft Windows operating system, including installing software, selecting menu items, clicking buttons, navigating files and folders.

You should read this guide for detailed API description of various use cases of ISOLAR before developing and executing scripts. The in-built tool help also contains description on how to write scripts in general.

2 API Documentation

The APIs are divided into categories of ARTOP Utils, AR Software Template, AR System Template and ISOLAR Utils. This entire chapter describes these APIs in detail and provides examples where ever necessary in Javascript. The chapter is also divided into the categories mentioned above

2.1 Java Scripting API's

The APIs are divided into categories of ARTOP Utils, AR Software Template, AR System Template and ISOLAR Utils. This entire chapter describes these APIs in detail and provides examples where ever necessary in Javascript. The chapter is also divided into the categories mentioned above

2.1.1 Categories of API's

ARTOP Utils

All the APIs that are related to ARTOP would be available in this section. These would generally include APIs to update the artop model, traversal, creating AUTOSAR Resources etc.

AR Software Template

These APIs are related to the AUTOSAR Software template and deal generally with Compositions, Components, Ports and Adapters for the component. This section is sub divided into sections mentioned above. APIs for Autoconnection, Adapter generation can be found here.

AR System Template

These APIs are related to the AUTOSAR System. APIs for AutoCleaup, AutoSwcToEcuMapping etc can be found in this section.

ISOLAR Utils

These APIs are provided by ISOLAR-A for use cases like reporting and querying relevant data like references of a particular element. These can be used for a variety of use cases.

2.1.2 ARTOP Utils

void getAllSplittableTypes(EObject parent, Class c);

Description :	
calculates list of split objects under the parent of type class c	
Return Type :	
List<EObject>	list of split objects
Parameters :	
parent	The AR Element which would have a containment relationship with the child AR Element.
c	Class type of the element of which splitable elements needs to be returned.

Example :
getAllSplittableTypes(system, Packages.autosar40.system.SystemMapping);

void addElement(EObject parent, EObject child, String role);

Description :	
Adds an Element to a parent EObject for the specified Role.	
Return Type :	
void	Method does not return any value.
Parameters :	
parent	The AR Element which would have a containment relationship with the child AR Element.
child	The AR Element which is to be contained by the Parent element.
role	Containment references are attributed with a roleName which would be the name of the collection of the contained elements. For more details of the Role of elements please refer ARTOP API for specific AR elements which are of interest.
Example :	
addElement(package, applicationDatatype, “elements”);	

Command addElementCommand(EObject parent, EObject child, String role);

Description :	
Adds an Element to a parent EObject for the specified Role.	
Return Type :	
void	Method returns the command which has to be executed to add the object to the parent.
Parameters :	
parent	The AR Element which would have a containment relationship with the child AR Element.
child	The AR Element which is to be contained by the Parent element.
role	Containment references are attributed with a roleName which would be the name of the collection of the contained elements. For more details of the Role of elements please

	refer ARTOP API for specific AR elements which are of interest.
Example :	
addElementCommand(package, applicationDatatype, “elements”);	

String createAutosarFile(String projectName,String fileName);

Description :	
Creates an Autosar file in the given ProjectName with the given filename. If the project does not exist, it would be created and file would be placed in it.	
Return Type :	
ARPackage	Returns the default ARPackage created
Parameters :	
projectName	Name of the Project in the Workspace
fileName	Name of the AR File to be created
Example :	
createAutosarFile(“ARProject”, “test.arxml”);	

String createAutosarFileAndAddPackages(String projectName,String filename, String regClass, List<EObject> listOfRootPackages);

Description :	
Creates an Autosar file in the given ProjectName with the given filename. If the project does not exist, it would be created and file would be placed in it.	
Return Type :	
String	Returns the default ARPackage created if the listOfRootPackages parameter is empty or null
Parameters :	
projectName	Name of the Project in the Workspace
fileName	Name of the AR File to be created
regClass	Registration class type
listOfRootPackages	List of root packages that needs to be added to created file
Example :	
createAutosarFileAndAddPackages (“ARProject”, “test.arxml”,”TDATA-ARA”,listOfRootPackages);	

IProject createAutosarProject(String projectName);

Description :	
Creates and Autosar Project with specified name.	
Return Type :	
IProject	Returns created AR Project. This is standard Eclipse IProject interface. Please refer Eclipse Core Resources IProject API for member details.
Parameters :	
projectName	The name of the project to be created as String.
Example :	
createAutosarProject("ARProject");	

void deleteFromModel(EObject object);

Description :	
Deletes an element from the AR Model. It removes all the references the element would have had from other AR elements. The object itself is removed from the parent containment and deleted.	
Return Type :	
void	Method does not return any value.
Parameters :	
object	The AR Element to be deleted.
Example :	
deleteFromModel(applicationDataType);	

Autosar40Factory getAutosar40FactoryInstance();

Description :	
Returns a factory Instance for creating ARElements.	
Return Type :	
Autosar40Factory	This Factory instance can be used for creation of AR Elements using a factory pattern. Please refer ARTOP Autosar40Factory API for member and method details.
Parameters :	
Example : getAutosar40FactoryInstance()	

EObject write(EObject object);

Description :
Updates the Autosar element in a write transaction. This means that all updates to the element's properties will be serialized to the arxml in one transaction

state. All updates to the ARTOP model are to be done in a Write Transaction. This is necessary for proper serialization of the model.	
Return Type :	
EObject	The Modifed Autosar Elment is returned after the transaction.
Parameters :	
object	The AR element which is to be updated.
Example :	
write(connector).setCategory("NEW");	

void removeElement(EObject parent, EObject child, String role);

Description :	
Removes an Element to a parent EObject for the specified Role.	
Return Type :	
void	Method does not return any value.
Parameters :	
parent	The AR Element which would have a containment relationship with the child AR Element.
child	The AR Element which is to be contained by the Parent element.
role	Containment references are attributed with a roleName which would be the name of the collection of the contained elements. For more details of the Role of elements please refer ARTOP API for specific AR elements which are of interest.
Example :	
removeElement(package, applicationDatatype, “elements”);	

Command removeElementCommand(EObject parent, EObject child, String role);

Description :	
Removes an Element to a parent EObject for the specified Role.	
Return Type :	
void	Method returns the command which has to be executed to remove the child from the parent.
Parameters :	
parent	The AR Element which would have a containment relationship with the child AR Element.

child	The AR Element which is to be contained by the Parent element.
role	Containment references are attributed with a roleName which would be the name of the collection of the contained elements. For more details of the Role of elements please refer ARTOP API for specific AR elements which are of interest.
Example :	
removeElementCommand(package, applicationDatatype, “elements”);	

Command setCommad(EObject owner, Object vaule, String role);

Description :	
set the owner's feature to the specified value.	
Return Type :	
void	Method returns the command which has to be executed to set the value to the owner
Parameters :	
owner	The AR Element which would have a containment relationship with the child AR Element.
value	Value to be set to the attribute of the owner
role	Containment references are attributed with a roleName which would be the name of the collection of the contained elements. For more details of the Role of elements please refer ARTOP API for specific AR elements which are of interest.
Example :	
setCommand(package, “value”, “elements”);	

Command getEditingDomain(EObject owner);

Description :	
Gives the transactional editing domain that manages the specified element.	
Return Type :	
void	Method returns the transactional editing domain that manages the owner.
Parameters :	
owner	The AR Element for which the editing domain is required to set the values to the feature
Example :	
getEditingDomain (ecuInstance);	

void set(EObject owner, Object vaule, String role);

Description :	
This creates a command to set the owner's feature to the specified value.	
Return Type :	
void	Method returns nothing.
Parameters :	
owner	The AR Element which would have a containment relationship with the child AR Element.
value	Value to be set to the attribute of the owner
role	Containment references are attributed with a roleName which would be the name of the collection of the contained elements. For more details of the Role of elements please refer ARTOP API for specific AR elements which are of interest.
Example :	
set (package, “value”, “elements”);	

CompoundCommand getCompoundCommand();

Description :	
This creates a new compound command	
Return Type :	
CompoundCommand	Returns the new compound command
Example :	
getCompoundCommand();	

2.1.3 AR Software

This module contains APIs that are related to AR Software Template

2.1.3.1 Adapter Generation

APIs for generating adapters.

- IStatus generateAdapter(IProject project,GSwComponentType componentOrCompositionName,String ecuInstanceName,String basePackageName,String outputFolder,Boolean closeOpenPorts,Boolean isSelfContained);

Description :
This API invokes Adapter Generation on a component or composition. The parameters to be passed are the component or composition name, the project name, the ecu instance name, the base package name, the output folder name, in case of composition if the adapter is to close open ports or simply create an adapter for the composition and if it needs to be self contained.

Return Type :	
IStatus	The status of the generation process. Please refer eclipse IStatus API for member details.
Parameters :	
project	The project in which the Adapters are to be generated.
componentOrComposition-Name	The Component or Composition object. Should be of type SwComponentType or subclasses.
ecuInstance	The name of the EcuInstance.
basePackageName	The base package for the adapters which will be generated as String.
outputFolder	The outFolder name where the generated files have to be kept.
closeOpenPorts	True if the open ports of the Composition are to be closed.
isSelfContained	True if if the generated adapter component(s) are to be self contained. All data related to adapter components will be self contained.
Example :	

- IStatus generateAdapterStringAPI(String project,String componentOrComposition-Name,String ecuInstance,String basePackageName,String outputFolder,boolean closeOpenPorts,boolean isSelfContained);

Description :	
This API invokes Adapter Generation on a component or composition. The parameters to be passed are the component or composition name, the project name, the ecu instance name, the base package name, the output folder name, in case of composition if the adapter is to close open ports or simply create an adapter for the composition and if it needs to be self contained. This is a string API that takes all the above parameters as String values.	
Return Type :	
IStatus	The status of the generation process. Please refer eclipse IStatus API for member details.
Parameters :	
project	Name of the project in workspace as a String
componentOrComposition-Name	Name of Component or Composition as a String
ecuInstance	Name of the EcuInstance
basePackageName	The base package for the adapters which will be generated as String.

outputFolder	The outFolder name where the generated files have to be kept.
closeOpenPorts	True if the open ports of the Composition are to be closed.
isSelfContained	True if if the generated adapter component(s) are to be self contained. All data related to adapter components will be self contained.
Example :	

2.1.3.2 Components

APIs related to Components –

- List getAllCompuMethodsUsedInComponent(AtomicSwComponentType component);

Description :	
Returns all the CompuMethods used in the Component. The compumethods that are referred in the Data types of the Port Interface elements would be considered.	
Return Type :	
List	Return a list of Compumethods. This is standard java list interface. Elements in the list are of type Compumethod. Please refer ARTOP Compumethod API for member details.
Parameters :	
component	The Component for which Compumethods are to be fetched.
Example :	

var compumethods = getAllCompuMethodsUsedInComponent (component);
var iterator = compumethods.iterator();
while(iterator.hasNext()){
var compumethod = iterator.next();
}	

- List getAllDataDictionaryElements(AtomicSwComponentType component);

Description :	
Returns all the Data Dictionary elements associated with a AtomicSwComponent. This is basically a collection of all the compumethods, datatypes and interfaces used in a component.	
Return Type :	
List	Return a list of DataDictionary Elements. This is standard java list interface. Elements in the list are of type

	Compumethod. Please refer ARTOP Compumethod API for member details.
Parameters :	
component	The Component for which DataDictionary elements are to be fetched.
Example :	
var elements = getAllDataDictionaryElements(component);	

- List getAllDatTypesUsedInComponent(AtomicSwComponentType component);

Description :	
Returns all the DataTypes used in the Component. These are the datatypes of the Variable Dataprototypes and parameter dataprototypes of interfaces referred by the ports of the component.	
Return Type :	
List	Return a list of Datatypes. This is standard java list interface. Elements in the list are of type AutosarDatatype or its subclasses. Please refer ARTOP AutosarDatatype API for member details.
Parameters :	
component	The Component for which Datatypes are to be fetched.
Example :	
var datatypes = getAllDatTypesUsedInComponent(component);	

- List getAllInterfacesUsedinComponent(AtomicSwComponentType component);

Description :	
Returns all the Interfaces used in the Component. These are basically all the interfaces referred by the ports of a component.	
Return Type :	
List	Return a list of Interfaces. This is standard java list interface. Elements in the list are of type PortInterface or its subclasses. Please refer ARTOP PortInterface API for member details.
Parameters :	
component	The Component for which Interfaces are to be fetched.
Example :	
var interfaces = getAllInterfacesUsedinComponent(component);	

- IStatus createIsolarCodeFrame("componentName",{log:logFilePath, preference:optionalFilePath});

Description :	
This API can be used for the generation of C files for the software components once the user has completed their configuration. The generated C file for a component is a template containing Runnable methods and RTE API calls based on the configuration of the RunnableEntity within the Component	
Return Type :	
IStatus	The status of the generation process. Please refer eclipse IStatus API for member details.
Parameters :	
component	The Component for which C file needs to be generated
preference	File containing all customization values and other required inputs
Example :	
createIsolarCodeFrame('WiperControl_100us',{log:'C:\Temp\newone.log',preference:"C:\temp\preferenceFile"});	

2.1.3.3 Composition

APIs related to AR Compostion -

- List getAllAssemblyConnectors(CompositionSwComponentType composition);

Description :	
Returns a list of all the Assembly Connectors contained in the Composition.	
Return Type :	
List	List of AssemblySwConnectors present under Composition. This is standard java list interface. Elements in the list are of type AssemblySwConnectors. Please refer ARTOP AssemblySwConnectors API for member details.
Parameters :	
composition	Composition object for which assembly connectors are to be fetched.
Example :	
var list = getAllAssemblyConnectors (composition);	

- List getAllDelegationConnectors(CompositionSwComponentType composition);

Returns all the Delegation Connectors in the Composition

Description :

Creates and Autosar Project with specified name.	
Return Type :	
List	List of DelegationSwConnectors present under Composition. This is standard java list interface. Elements in the list are of type DelegationSwConnector. Please refer ARTOP DelegationSwConnector API for member details.
Parameters :	
composition	Composition object for which delegation connectors are to be fetched.
Example :	
var list = getAllDelegationConnectors (composition);	

- void validateComposition(CompositionSwComponentType composition);

Description :	
Creates and Autosar Project with specified name.	
Return Type :	
void	Method does not return any value. Populates errors to error log.
Parameters :	
composition	Composition object which has to be validated.
Example :	
validateComposition(composition);	

2.1.3.4 Connections

APIs related to Connections -

- void autoAssemblyConnection(CompositionSwComponentType composition);

Description :	
Performs AutoAssembly Connection for the composition. Naming Rules and other settings will be taken from the Preferences.	
Return Type :	
void	Method does not return any value.
Parameters :	
composition	The composition for which assembly connection is to be performed
Example :	
autoAssemblyConnection(composition);	

- void autoDelegationConnection(CompositionSwComponentType composition);

Description :	
Performs AutoDelegation Connection for the composition. Naming Rules and other settings will be taken from the Preferences.	
Return Type :	
void	Method does not return any value.
Parameters :	
composition	The composition for which delegation connection is to be performed
Example :	
autoDelegationConnection(composition);	

- void deleteErroneousConnections(CompositionSwComponentType composition);

Description :	
Deletes all the erroneous connections in composition. The connections which are not valid will be removed from the model.	
Return Type :	
void	Method does not return any value.
Parameters :	
composition	The composition from which erronoues connections are to be deleted.
Example :	
deleteErroneousConnections(composition);	

- List getAllErroneousConnections(CompositionSwComponentType composition);

Description :	
Returns a list of erroneous connections in Composition.	
Return Type :	
List	List of erroneous connections in the Composition. Elements in the list are of type SwConnector and its subclasses. This is standard java list interface.
Parameters :	
composition	The composition from which connectors are to be fetched.
Example :	
var errorConnections = getAllErroneousConnections(composition);	

2.1.3.5 Ports

APIs related to Ports

- List findAllOpenPorts(SwComponentType composition);

Description :	
Returns a list of open ports in the composition. These are ports which are not connected to other ports through connections either within or outside the composition.	
Return Type :	
List	List of open ports. Elements in the list are of type PortPrototype and its subclasses. Please refer ARTOP PortPrototype API for member details.
Parameters :	
composition	Composition for which open ports are to be found.
Example :	
var listofPorts = findAllOpenPorts(composition);	

2.1.4 AR System

This module contains APIs that are related to AR System Template

- List autoCleanUpAllMappings(System system);

Description :	
Deletes the erroneous mappings in the System.	
Return Type :	
List	Returns a list of deleted erroneous mappings under the System ARElement.
Parameters :	
system	The System ARElement under which mappings are to be inspected.
Example :	
var list = autoCleanUpAllMappings(system);	

- void generateSwcToEcuMappings(System system,Boolean all,List componentsList);

Description :	
Generates Swc Ecu Mappings for the system with a check whether for all components or list of selected Components.	
Return Type :	
void	Method does not return any value.
Parameters :	

system	The system element for which SwcToEcuMappings have to be generated.
all	True if mappings are to be generated for all ComponentPrototypes
componentsList	List of componentPrototypes that are to be mapped.

2.1.5 ISOLAR Utils

This module contains Utils of ISOLAR-A which are generally useful for Model Update/Reporting and other use cases.

- void createUniqueShortName(EObject object);

Description :	
This API is used to generate an unique shortname for an EObject. The EObject is passed as a parameter and for this EObject a unique shortname will be generated. Thus generated short name will be set back to the EObject. This is useful when a new AR element is created from the AR Factory instance using the getAutosar40FactoryInstance() API	
Return Type :	
void	Method does not return any value
Parameters :	
object	AR Element for which unique shortname has to be generated and set.
Example :	

- List getAllInstancesAsList(IProject project,Class class,Boolean exactMatch);

Description :	
This API is used to fetch all the instances of given ClassType. This is to be used only if the ClassType is a Packageable element(All AR elements that can come directly as children to ARPackages are called Packageable Elements).	
Return Type :	
List	List of instances of the given class type. This is standard java list interface. Elements in the list are of type provided as argument.
Parameters :	
project	AR Project in which instances are to be searched. This is of type - Standard eclipse IProject interface.
class	Class type of instances which have to be searched.

exactMatch	Boolean whether exact match has to be considered and not the child inheritance classes from the specified class as parameter.
Example :	
var list = getAllInstancesAsList (project, applicationDatatype.class, true);	

- List getAllPackageableElements(String projectName);

Description :	
This API will take project name as a parameter. By using this parameter it the project will be identified. Thus identified project will be searched for the packageable elements. All the packageable elements found in the project will be returned as a list of PackageableElement(s). All AR elements that can come directly as children to ARPackages are called Packageable Elements.	
Return Type :	
List	List of Packageable elements present in project. This is standard java list interface. Elements in the list have classtypes derived from Packageable Element of ARTOP.
Parameters :	
projectName	String Project Name from where the elements have to fetched
Example :	
var elements = getAllPackageableElements(“test_ARProject”);	

- List getAllReferncesInProject(EObject object,Boolean fetchFromReferencedProject);

Description :	
This API is used to fetch all the references of an EObject in a project. If the boolean value fetchFrmReferencedPrj is true then the references from the linked projects are also fetched as a list of EObjects.	
Return Type :	
List	List of all elements that refer the object passed as parameter. This is standard java list interface. Elements in the list can be of various types and can be found by type checking.
Parameters :	
object	The element whose references have to be found.
fetchFromReferencedProject	True if referred Projects also have to be searched for references.
Example :	
var references = getAllReferencesInProject(applicationDataType, true);	

- List getAllReferncesInProject(EObject object,Collection parents);

Description :	
This API is used to fetch all the references of an EObject in a collection and returns as a list of EObjects. Returns all references of the particular object for the elements contained in the list as parameter.	
Return Type :	
List	List of all elements that refer the object passed as parameter. This is standard java list interface. Elements in the list can be of various types and can be found by type checking.
Parameters :	
object	The element whose references are to be found.
parents	Collection of parents under which the references have to be found. This can be a list of AR Elements, their children would be searched for references. This is different from the previous API as the previous API searches the entire Project.
Example :	
var references = getAllReferencesInProject(applicationDataType, parentsList);	

- List getAllResources(IProject project);

Description :	
Returns all resources in a Project as a list. These can be the Arxml file or any other resources in the project. All handles returned in the List are of type - Eclipse Resources	
Return Type :	
List	List of resources present in the project. This is standard java List interface. Elements in list are of type Eclipse Core Resources. Please refer Core Resources API for member details.
Parameters :	
project	The Project in workspace for which the resources are to be found.
Example :	
var resources = getAllResources(arProject);	

- List getAllRootArPackages(String project);

Description :
This API is usefull in fetching all the first level ARPackages from a project. The project name is passed as a parameter and this parameter is used to identity the project and from this project all the first level ARPackages will be fetched. All the fetched ARPackages will be returned as a list of EObjects.

Return Type :	
List	List of ARPackages which are at the root of the AR Project. This list will contain elements of type ARPackage. This the standard java List interface. Please refer API documentation of ARTOP AR40 ARPackage for member details of ARPackage
Parameters :	
project	String project name of the project present in Workspace
Example :	
var rootPackages = getAllRootPackages("test_ARProject");	

- ARPackage getParentARPackage(EObject object);

Description :	
This API can be used to fetch the parent ARPackage of an EObject. The EObject is passed as a parameter and by using this EObject the parent ARPackage is fetched.	
Return Type :	
ARPackage	This is the base class for autosar ARPackage. The API returns the ARPackage which is the parent of the AR Element. Please refer API documentation of ARTOP AR40 ARPackage for member details
Parameters :	
object	The element for which the parent ARPackage is to be found.
Example :	
var arPackage = getParentARPackage(applicationDataType);	

- IProject getProject(Object object);

Description :	
This API returns the Autosar project in which the element is present in.	
Return Type :	
IProject	The IProject the element is present in. This is standard eclipse IProject interface. Please refer API documentation of org.eclipse.core.resources.IProject for member details.
Parameters :	
object	The element for which Project has to be found.
Example :	
var project = getProject(applicationDataType);	

- List getSiblings(EObject object);

Description :	
This API is used to fetch all the siblings of an EObject. It will fetch all the contents from the EObject and checks for the equality of EClass object. Siblings – Elements of the same parent, having same type.	
Return Type :	
List	The list of siblings of the passed object. If object does not have sibblings then list size will be 0. Iterator has to be used to iterate over the sibling elements. This the standard java List interface.
Parameters :	
object	Element for which siblings are to be found.
Example :	
var siblings = getSiblings(applicationDataType);	

- boolean isNotNullProxy(EObject object);

Description :	
This API will check for whether the object is not null and not proxy. It returns true if the object is not null and is not a proxy, false otherwise.	
Return Type :	
boolean	Boolean true or false depending on whether element is Null/Proxy
Parameters :	
object	The element that is being checked
Example :	
var booleanValue = isNotNullProxy(applicationDataType);	

- boolean isParamDefElement(Object object);

Description :	
This API is used to check whether the object is of type ParamDef or not. It returns true, if it is a ParamDef element, false otherwise.	
Return Type :	
boolean	Boolean value true or false depending on whether element is paramDef element or not
Parameters :	
object	The element that is being checked
Example :	

var booleanValue = isNotNullProxy(applicationDataType);

- void setNewUUID(EObject object);

Description :	
This API is used to create a new UUID to the newly created EObject. EMF EObjects are generated with a unique ID which differentiate them from others. This is required only when new objects are created with the AutosarFactory Instance.	
Return Type :	
void	Method does not return any value
Parameters :	
object	The Identifiable object whose Timestamp must be set.
Example :	
setNewUUID(system);	

- void setTimeStamp(EObject object);

Description :	
This API is used to set a new timestamp to the newly created EObject. EMF EObjects are generated with a unique timestamp which differentiate them from others. This is required only when new objects are created with the AutosarFactory Instance.	
Return Type :	
void	Method does not return any value
Parameters :	
object	The Identifiable object whose Timestamp must be set.
Example :	
setTimeStamp(system);	

- boolean isBamfExecution(IScriptEngine engine);

Description :	
This API is used to check is the execution of the script is from Bamf action or not.	
Return Type :	
void	Method return true if execution from Bamf else false
Parameters :	
engine	Script engine that is being used to run these scripts.
Example :	
isBamfExecution(scriptEngine);	

- boolean isCliExecution(IScriptEngine engine);

Description :	
This API is used to check is the execution of the script is from CLI or not.	
Return Type :	
void	Method return true if execution from CLI else false
Parameters :	
engine	Script engine that is being used to run these scripts.
Example :	
isCliExecution(scriptEngine);	

- List<String> filesToRegisterInBamf(IScriptEngine engine);

Description :	
This API is used to register the generated files out of scripts by Bamf. Registration class needs to be specified in the bamf output data	
Return Type :	
List<String>	Method returns List of String to which relative file path can be added that needs to be registered
Parameters :	
engine	Script engine that is being used to run these scripts.
Example :	
filesToRegisterInBamf(scriptEngine);	

2.2 Java Action APIs

This module contains Utils of ISOLAR-A which are generally useful for Model Update/Reporting and other use cases.

2.2.1 Categories of API's

- ASWLibrary Utils

  These APIs are related to invoking a feature in CLI Mode.So with these api's different features like Ecu Extract, Auto Connection etc can be invoked using Java Action.

- JavaActionUtility

  These APIs deal with read data,write data,Log the data ect in Csv and Excel format.

- ASWModel

  These APIs deal with model,required input element etc.

- Artop Utils

  All the APIs that are related to ARTOP would be available in this section. These would generally include APIs to update the artop model, traversal, creating AUTOSAR Resources etc.

2.2.2 ASWLibrary Utils

- boolean generateAdapterGeneration(String compName, String additionalFile , ILog log);

Description :	
Auto generates the adapter for the given Component or Composition.	
Return Type :	
boolean	Returns true if adapters are generated successfully else false
Parameters :	
compName*	Short name of the Component or Composition
additionalFile	Additional preference input file path. (both relative and absolute path is supported)
log	Console logger available in input context
Example :	
generateAdapterGeneration("WiperControl_100us","additionalInputFile.txt" , consoleLogger);	

- boolean deleteErroneousConnection(String composition,String logFile, String additionalFile , ILog log);

Description :	
Deletes if there are any Erroroneous connections in the Composition.	
Return Type :	
boolean	Returns true if the execution was successful else false
Parameters :	
composition	Absolute Qualified name of the Composition for which the erroneous connections shall be deleted.
logFile	Optional command to log results in the specified file.(both relative and absolute path are supported)
additionalFile	Additional preference input file path. (both relative and absolute path is supported)
log	Console logger available in input context
Example :	
deleteErroneousConnection("/WiperControl_Composition/WiperControl_SystemComposition"," additionalInputFile.txt" , consoleLogger);	

- boolean autoConfigureComposition(String swComposition, String logFile, String additonalOptionalFile, ILog consoleLog);

Description :	
Auto configures a composition by instantiating components, generating possible connections, mapping components to ECUs, deleting erroneous connections and generating ECUExtract	
Return Type :	
boolean	Returns true if the execution was successful else false
Parameters :	

swComposition	Short name of the Composition
logFile	Optional command to log results in the specified file.(both relative and absolute path are supported)
addtionalFile	Additional preference input file path. (both relative and absolute path is supported)
log	Console logger available in input context
Example :	
autoConfigureComposition ("WiperControl_SystemComposition",true,true,true,true,true,true,true, "WiperControl","newconfig.txt","Split.arxml",logFile.txt”,"preferenceFile.txt", consoleLogger);	

- boolean autoSignalMapping(String system, String mappingType, String additionalFile, ILog log);

Description :	
Maps the Signal/SignalGroup to a DataElement in the system	
Return Type :	
boolean	Returns true if the execution was successful else false
Parameters :	
system	Absolute Qualified name of the System in which mappings needs to be generated and related RootSwComposition will be considered for generating mappings
additionalFile	Additional preference input file path. (both relative and absolute path is supported)
mappingType	Mapping type that needs to be generated. Possible types: SR – SenderReceiverToSignalMapping TR –TriggerToSignalMapping CD – SenderReceiverCompositeDataToSignalMapping CS - ClientServerToSignalMapping
log	Console logger available in input context
Example :	
autoSignalMapping("/WiperControl_System_EcuExtract/EXTR_WiperControl","SR","filepath.txt" , consoleLogger);	

- boolean autoSwcToEcuMapping(String system, String composition, String ecu, String additionalFile, String logFile, ILog log);

Description :	
Maps the software components to ECU’s	
Return Type :	
boolean	Returns true if the execution was successful else false
Parameters :	
system	Absolute Qualified name of the System
composition	Absolute Qualified name of the Composition

ecu	Absolute Qualified name of the ECU Instance.
additionalFile	Additional preference input file path. (both relative and absolute path is supported)
logFile	Optional command to log results in the specified file.(both relative and absolute path are supported)
log	Console logger available in input context
Example :	
AutoSwcToEcuMapping("/WiperControl_System_EcuExtract/EXTR_WiperControl","/WiperControl_Composition/WiperControl_SystemComposition ","/WiperControlCommunicationMatrix/Pkg_Ecu/WiperControl","filepath.txt" , consoleLogger);	

- boolean performValidation(String profile, List<EObject> scope, String version, String logFile, ILog log);

Description :	
Performs Autosar validation on the Project	
Return Type :	
boolean	Returns true if the execution was successful else false
Parameters :	
profile	Validation Profile(System,Software,Ecu,Dext,Schema,BSWMDT or BSW)
scope	The list of ArElements on which the validation to be performed
version	Autosar release version(4x or specific version could be mentioned)
logFile	Optional command to log results in the specified file.(both relative and absolute path are supported)
log	Console logger available in input context.
Example :	
performValidation ("system","",ecuList ,"4x","log.txt", consoleLogger);	

- boolean autoConnect(String connectionType, String compsoition, String splitfile, String logFile, String additionalFile, ILog consoleLogger);

Description :	
Auto-generates the connections within a given composition	
Return Type :	
boolean	Returns true if the execution was successful else false
Parameters :	
connectionType	Type of the connection(assembly,delegation,passthrough)
compsoition	Absolute Qualified name of the Composition
splitfile	This is specified to split the generated connections in the specified arxml file path.

logFile	Optional command to log results in the specified file.(both relative and absolute path are supported)
additionalFile	Additional preference input file path. (both relative and absolute path is supported)
log	Console logger available in input context.
Example :	
autoConnect("assembly"," /WiperControl_Composition/WiperControl_SystemComposition ", consoleLogger);	

- boolean createEcuExtract(String ecu, String logFile, String additionalFile, ILog log);

Description :	
Extracts ECU specific information from the System Configuration Description and also transforms the hierarchical System to a Flat view	
Return Type :	
boolean	Returns true if the execution was successful else false
Parameters :	
ecu	Short name of the ECU Instance
logFile	Optional command to log results in the specified file.(both relative and absolute path are supported)
additionalFile	Additional preference input file path. (both relative and absolute path is supported)
log	Console logger available in input context.
Example :	
createEcuExtract("WiperControl","filepath.txt" , consoleLogger);	

- boolean importRTTM(String ecucvalue, String mappingType, String input, String log-File, String additionalFile, ILog log);

Description :	
Imports the RRTM configuration from excel file	
Return Type :	
boolean	Returns true if the execution was successful else false
Parameters :	
ecucvalue	Short name of the ecucValueCollection which contains Rte and Os configurations
mappingType	Type of mapping (Entity2Task, Event2Task)
input	Excel file path where Runnable| event to task mapping are configured
logFile	Optional command to log results in the specified file.(both relative and absolute path are supported)
additionalFile	Additional preference input file path. (both relative and absolute path is supported)
log	Console logger available in input context.
Example :	
importRTTM ("WiperControl","Event2Task","file.xlxs","filepath.txt" , consoleLogger);	

- boolean autoSwcToImplMapping(String system, boolean doCreateNewMappings, String logFile, ILog consoleLog);

Description :	
Auto generates SwComponent To Implementation mapping	
Return Type :	
boolean	Returns true if the execution was successful else false
Parameters :	
System	Short name or the Absolute Qualified name of the System
doCreateNewMappings	Creates new SwcToImpl Mappings. If it is not specified, the existing mappings will be overridden
logFile	Optional command to log results in the specified file.(both relative and absolute path are supported)
log	Console logger available in input context.
Example :	
autoSwcToImplMapping ("/WiperControl_System/WiperControl_System", true, "autoSwc2Impllog.log", this.log);	

- boolean generateSystemExtract(final String ecuInstance, final String system, final String targetProject, String logFile, String additonalOptionalFile, final ILog consoleLog);

Description :	
Generates SystemExtract configuration for the selected EcuInstances & System	
Return Type :	
boolean	Returns true if the System extract generation was successful else false
Parameters :	
Ecuinstance	Short names of all EcuInstances (separated by comma) that needs to be extracted
System	Short name or Absolute Qualified name of the System from which the configuration needs to be extracted
targetProject	Target project where SystemExtract needs to be generated. Could also specify absolute path if extract has to be generated outside the workspace
logFile	Optional command to log results in the specified file.(both relative and absolute path are supported)
additonalOptionalFile	Additional preference input file path.
consoleLog	Console logger available in input context.
Example :	

generateSystemExtract("WiperControl,Debug","WiperControl_System\","SystemExtract","systemExtractLog.log" ," additionalInputFile.txt" , this.log);

- boolean generateComMatrix (final String ecuInstance, final String targetProject, String logFile, String additonalOptionalFile, final ILog consoleLog);

Description :	
Extracts Communication matrix configuration for the selected EcuInstances	
Return Type :	
boolean	Returns true if the Communication matrix configuration generation was successful else false
Parameters :	
Ecuinstance	Short names of all EcuInstances (separated by comma) that needs to be extracted
targetProject	Target project where CommunicationMatrix Extract needs to be generated. Could also specify absolute path if extract has to be generated outside the workspace
logFile	Optional command to log results in the specified file.(both relative and absolute path are supported)
additonalOptionalFile	Additional preference input file path.
consoleLog	Console logger available in input context.
Example :	
generateComMatrix("WiperControl,Debug",commatrix","commatrixLog.log" ," additionalInputFile.txt" , this.log);	

- List<String> delegateOpenPorts(String composition, String prefix, String suffix, ILog consoleLog)

Description :	
Extracts Communication matrix configuration for the selected EcuInstances	
Return Type :	
boolean	Returns the message whether the open ports are delegated or not.
Parameters :	
composition	Short name or the Absolute Qualified name of the composition
prefix	Prefix to be appended to the name of the port
suffix	suffix to be appended to the name of the port
consoleLog	Console logger available in input context.
Example :	
generateComMatrix("WiperControl,Debug",commatrix","commatrixLog.log" ," additionalInputFile.txt" , this.log);	

- boolean generateCodeFrame(String component, String destinationFolder , String additionalFile, ILog log);

Description :	
Generate the code frame file for given component.	
Return Type :	
boolean	Returns true if the execution was successful else false
Parameters :	
component	Absolute Qualified name of the component.
destination folder	Destination folder where code frame files will be created.
additionalFile	Additional preference input file path. (both relative and absolute path is supported)
log	Console logger available in input context
Example :	
generateCodeFrame("/WiperControl_2ms/ WiperControl_2ms ","Component CodeFrame/","filepath.txt" , consoleLogger);	

- boolean performAutoCleanUp(String system, String logFile, String OptionalFilePath, ILog consoleLog);

Description :	
Removes the erroneous and proxy elements	
Return Type :	
boolean	Returns true if the execution was successful else false
Parameters :	
system	Absolute Qualified name of the system.
destination folder	Destination folder where code frame files will be created.
additionalFile	Additional preference input file path. (both relative and absolute path is supported)
logFile	Optional command to log results in the specified file.(both relative and absolute path are supported)
log	Console logger available in input context
Example :	
performAutoCleanUp ("/WiperControl_System/WiperControl_System","filepath.txt" , consoleLogger);	

- boolean aswAutoGen(String namessOfEcus, String logFile, String additonalOptionalFile, ILog consoleLog);

Description :	
Auto generates the component and composition	
Return Type :	
boolean	Returns true if the execution was successful else false
Parameters :	
namessOfEcus	Absolute qualified names or short names of Ecu's with comma separation
logFile	Optional command to log results in the specified file.(both relative and absolute path are supported)
additionalFile	Additional preference input file path. (both relative and absolute path is supported)
log	Console logger available in input context
Example :	
aswAutoGen ("WiperControl","filepath.txt" , consoleLogger);	

- boolean reportGeneration(String reportID, String outputFilepath, ILog consoleLog, List<String>inputFiles);

Description :	
Generate the excel report with the given report id	
Return Type :	
boolean	Returns true if the execution was successful else false
Parameters :	
reportID	Each report id has different templates i.e possible report id's: 1 : its generates DataTypes information 2 : its generates Composition information 3 : its generates EcuExtract information 4 : its generates Component information 7 : its generates EcuExtractCombot information 8 : its generates ARXML DiffMerge information -1 : its generates complete information
output-Filepath	File path where the report needs to be generated(default path is C:\Temp\Gen-Report.xls)

log	Console logger available in input context
inputFiles	[Conditionally Optional] Comma separated arxml absolute file paths. It is mandatory when the --reportid given with the value "8"
Example :	
reportGeneration ("1","" , consoleLogger);	

- boolean createClusterExtract (final Object clusternames, final String system, String logFile, String additonalOptionalFile, final ILog consoleLog);

Description :	
Extracts Communication matrix configuration for the selected EcuInstances	
Return Type :	
boolean	Returns true if the Communication matrix configuration generation was successful else false
Parameters :	
cluster	[Optional] Shortname of cluster that needs to be extracted.If no cluster names are provided,it will take all possible clusters from the system.
system	[Optional] Shortname of System from which cluster needs to be extracted.if system name is not provided it will consider all system from the project.
logFile	Optional command to log results in the specified file.(both relative and absolute path are supported)
additonalOptionalFile	Additional preference input file path.
consoleLog	Console logger available in input context.
Example :	
createClusterExtract ("ClusterAHB ",”System_0","clusterExtract.log" ," additionalInputFile.txt" , this.log);	

- boolean ldfImport(final String file, final String output, final String variant, final String ecu, final String logFile, final String additonalOptionalFile, final ILog consoleLog);

Description :	
This will import ldf file into the project and generates ARXML file	
Return Type :	
boolean	Returns true if the ldf import configuration was successful else false
Parameters :	
file	[Mandatory] Name of the LDF File to be imported.
output	[Optional] The output arxml file to be generated.
ecu	[Optional] The ecus to be included.

variant	[Optional] The variants to be included.
logFile	Optional command to log results in the specified file.(both relative and absolute path are supported)
additonalOptionalFile	Additional preference input file path.
consoleLog	Console logger available in input context.
Example :	
ldfImport (“LDFFile.ldf” ,”\\dext.arxml","”, null, “logfile”, " additionalInputFile.txt" , this.log);	

- boolean fibexImport (final String file, final String output, final String additonalOptionalFile, final ILog consoleLog);

Description :	
This will import fibex file into the project and generates ARXML file	
Return Type :	
boolean	Returns true if the fibex import configuration was successful else false
Parameters :	
file	[Mandatory] Name of the fibex File to be imported.
output	[Optional] The output arxml file to be generated.
additonalOptionalFile	Additional preference input file path.
consoleLog	Console logger available in input context.
Example :	
fibexImport (“FibexFile.xml” ,”test.arxml“, " additionalInputFile.txt" , this.log);	

- boolean exportComponent(final String componentList, final String additonalOptionalFile, final String logFile,final ILog consoleLog);

Description :	
Exports the components specified from the system	
Return Type :	
boolean	Returns true if the execution was successful else false
Parameters :	
componentList	Short name of the Components
logFile	Optional command to log results in the specified file.
additonalOptionalFile	Additional preference input file path.
consoleLog	Console logger available in input context.
Example :	
exportComponent(\"WiperControl_2ms\",\"filepath.txt\" , consoleLogger);	

- boolean exportComposition (final String composition, final String additonalOptionalFile, final String logFile,final ILog consoleLog);

Description :	
Exports the compositions specified from the system	
Return Type :	
boolean	Returns true if the execution was successful else false
Parameters :	
composition	Short name of the Composition
logFile	Optional command to log results in the specified file.
additonalOptionalFile	Additional preference input file path.
consoleLog	Console logger available in input context.
Example :	
exportComposition(\"WiperControl_2ms\",\"filepath.txt\" ,"", consoleLogger);	

- boolean exportvariant (final String destination, final List<EObject> predefinedVariants, final String additionalFile, final ILog consolelog);

Description :	
Resolves the input project for the given list of predefined variants or the predefined variants configured in EcucVariantResolver and exports the actively resolved AR elements as a new autosar project in the given destination location.	
Return Type :	
boolean	Returns true if the execution is successful otherwise it returns false.
Parameters :	
destination	[Mandatory] The absolute path of directory to which the actively resolved AR elements will be exported as new autosar project
predefinedVariants	[Optional] Either the EcucVariantResolver or the list of predefined variants to resolve the input project.
additionalFile	[Optional] Additional preference input file path which contains the optional parameters which could be needed during execution. (both relative and absolute path is supported)
consolelog	Console logger available in input context.
Example :	
exportvariant (“c:\temp\ExportVariant” , Variant_0, " additionalInputFile.txt" , this.log);	

- boolean dbcImport (final String file, final String output, final String ecu, final String skipframes,final String mapping, final String logFile, final String additonalOptionalFile, final ILog consoleLog)

Description :

Imports the DBC file into the project and creates the elements as required.	
Return Type :	
boolean	Returns true if the execution was successful else false
Parameters :	
File	Name of the DBC File to be imported
output	The output arxml file to be generated
ECU	The ECU's to be included
skipframes	The frames that need to be skipped during the import
mapping	The mapping file that is used for the import
logFile	Optional command to log results in the specified file.
additonalOptionalFile	Additional preference input file path.
Example :	
dbcImport("Sample.dbc","DecSys.arxml" ,"WiperControl","",null,null," additionalInputFile.txt" , "consoleLogger");	

2.2.3 JavaActionUtility

- JavaActionLogFileGenerartor getLog(String filepath);

Description :	
It helps us to log the provided data into the txt file specified.	
Return Type :	
JavaActionLogFileGenerartor	Returns the object of the class JavaAction-LogFileGenerartor where it has different API's to log the information.
Parameters :	
filepath	Name of the log txt file to be created. (it supports both absolute and relative path of file)
Example :	
getLog ("TestLog.txt");	

- JavaActionCSVGenerator getCSV(String filepath, String... headers)

Description :	
It helps us to write the provided data into the CSV file specified.	
Return Type :	
JavaActionCSVGenerator	Returns the object of the class JavaAc-tionCSVGenerator where it has different API's to populatethe data in CSV format.
Parameters :	
filepath	Name of the CSV file to be created.

headers[]	Headers of the CSV file
Example :	
getCSV (“EmployeeDetails.csv”,”EmpName”,”NTID”,”Designation”);	

- ReadCSVFile readCSVFile(String filePath)

Description :	
It helps us to read the data from the CSV file specified.	
Return Type :	
ReadCSVFile	Returns the object of the class ReadCSVFile where it has API’s to read data from a CSV file.
Parameters :	
filepath	Name of the CSV file from which the the data has to be read.
Example :	
readCSVFile (“EmployeeDetails.csv”)	

- ReadExcelData readExcelFile(String filepath)

Description :	
It helps us to read the data from the excel file specified.	
Return Type :	
ReadExcelData	Returns the object of the class ReadExcelData where it has API’s to read/fetch data from an excel file.
Parameters :	
filepath	Name of the Excel file from which the the data has to be read/fetched
Example :	
readExcelFile (“EmployeeDetails.xlsx”)	

- WriteToExcelFile writeToExcelFile(String filepath)

Description :	
It helps us to write the data into the excel file specified.	
Return Type :	
WriteToExcelFile	Returns the object of the class WriteToExcelFile where it has API’s to write the data into the specified file.
Parameters :	
filepath	Name of the Excel file in to which the data has to be written

Example :
writeToExcelFile (“EmployeeDetails.xlsx”)

- Int executeExeFile(ILog consoleLog, String... cmd)

Description :	
It execute the “exe” files with the given parameters	
Return Type :	
Int	Returns 0 if the exe file execution is successful
Parameters :	
consoleLog	logs the output of the EXE application to the Console.
Cmd[]	An array of inputs i.e EXE FilePath, OutputFolder etc
Example :	
executeExeFile(log, “helloworld.exe”);	

2.2.4 ASWModel

- List<Object> getInputElement(boolean doGiveMergedElement);

Description :	
Selected element in UI or CLI will be return by this API in java file	
Return Type :	
List<Object>	Returns selected inputElement
Parameters :	
doGiveMergedElement	Passing true is same as invoking getInputElement(), passing false would return the actual element selected input element
Example :	
getInputElement(false);	

- List<Object> getInputElement()

Description : Selected element in UI or CLI will be return by this API in java file. This will merge all the split elements and return as a merged element	
Return Type :	
List<Object>	Returns the inputElement
Example :	
getInputElement();	

- IProject getProject();

Description :	
Return the project on which java action is executed.	
Return Type :	
IProject	Returns the IProject on which java action is executed
Example :	
getProject();	

- List<T> getElements(final Class<T> type)

Description :	
It gives the merged elements of the class type specified from the project on which java action is executed.	
Return Type :	
List<T>	Returns the list merged element
Parameters :	
type	Name of the inputElement class
Example :	
getElements (EcuInstance.class);	

- <T extends EObject> List<T> getUnmergedElements(final Class<T> type)

Description :	
It gives the unmerged elements of the class type specified from the project on which java action is executed.	
Return Type :	
List<T>	Returns the list of unmerged elements
Parameters :	
type	Class type of which elements needs to be fetched and returned
Example :	
getUnmergedElements (EcuInstance.class);	

- <T extends EObject> T getElement(final String fqn, final Class<T> type)

Description :	
It gives the merged instance of the class type specified with particular FQN.	
Return Type :	
EObject	Returns the merged instance the EObject
Parameters :	
fqn	Absolute Qualified path of the element to be returned

type	Class type of the element that needs to be fetched and returned
Example :	
getElement("/WiperControlCommunicationMatrix/Pkg_Ecu/WiperControl","EcuInstance.class");	

- <T> List<T> getUnmergedElements (String fqn, Class type);

Description :	
It gives the unmerged instance of the class type specified with particular FQN.	
Return Type :	
List<T>	Returns the list of actual instance of the EObject
Parameters :	
fqn	Absolute Qualified path of the element to be returned
type	Class type of the element that needs to be fetched and returned
Example :	
getUnmergedElements("/WiperControlCommunicationMatrix/Pkg_Ecu/WiperControl","EcuInstance.class");	

- <T extends EObject> T write(T EObject)

Description :	
It gives the modifiable instance of the given object. Note: only attribute level changes could be done with this.	
Return Type :	
EObject	Returns the modifiable instance of the passed object
Parameters :	
EObject	Element object on which the write action to be performed.
Example :	
write(ecuInstance);	

- ArtopUtils getArtopUtils()

Description :
It gives the ArtopUtils instance which contains all the required APIs
Return Type :

ArtopUtils	Returns the utility
Example :	
getArtopUtils();	

- ASWLibraryUtils getASWUtils()

Description :	
It gives the ASWLibraryUtils instance which contains all the required APIs	
Return Type :	
ASWLibraryUtils	Returns the utility
Example :	
getASWUtils ();	

- Map<String, String> parsePropertyFile(final String relativeFilePath, final String key-ValueSeperator);

Description :	
It parses the property file given and returns the map with key and value.	
Return Type :	
Map<String, String>	Returns the Map of key value pairs
Parameters :	
relativeFilePath	Name of the file to be parsed.
keyValueSeperator	Separator which separates the key and values
Example :	
parsePropertyFile(“pasefile.txt”, “=”);	

- Map<String,List<String>> parsePropertyFile(final String relativeFilePath, final String keyValueSeperator, final String valueSeperator);

Description :	
It parses the property file given and returns the map.	
Return Type :	
Map<String, List<String>>	Returns the Map of key & list of values associated with the key.
Parameters :	
relativeFilePath	Name of the file to be parsed.
keyValueSeparator	Separator which separates the key and values
valueSeparator	Separator which separates the list value.
Example :	

```
parsePropertyFile("pasefile.txt", "=","","");
```

- List<Resource> getAllResources();

Description :	
It returns all the autosar resources files in the given project.	
Return Type :	
List<Resource>	returns list all the autosar resources within given project
Example :	
getAllResources();	

- O updateModel(final Function<I, O> function, final I object)

Description :	
Any model update that needs to be performed should be included in a model update custom class (with method **public** IStatus method(CustomClass object)) and the object needs to be passed in the updateModel API.	
Return Type :	
o	Returns the function
Parameters :	
function	Name of the file to be parsed.
object	Separator which separates the key and values
Example :	

```
{

ModelUpdate myParameter = new ModelUpdate(model);
        model.updateModel(myParameter::method, myParameter);
}

class ModelUpdate {
    ASWModel model;

    public ModelUpdate(ASWModel model) {
        this.model = model;
    }

    public ASWModel getParameter() {
        return model;
    }

    /**
```

```
 * Method to update model
 *
 * @param object
 * @return
 */
public IStatus method(ModelUpdate object) {

Object element = this.model.getInputElement().get(0);

//creating new PPort
    PPortPrototype pPortPrototype = this.model.getArtopUtils().createAutosarElement(PPortProto-
type.class);
    pPortPrototype.setShortName("PPort_0");

    //Setting existing Provided Interface
     pPortPrototype.setProvidedInterface(this.model.getElement("/PortInterfaces/Torque_SRI", Sender-
ReceiverInterface.class));

    if(element instanceof ApplicationSwComponentType)
((ApplicationSwComponentType)element).getPorts().add(pPortPrototype);

   return Status.OK_STATUS;
}
}
```

2.2.5 Artop Utils

- EObject createAutosarElement(Class classType);

Description :	
Creates Autosar element for the specified class type.	
Return Type :	
EObject	Returns the created Autosar element
Parameters :	
Class	Class type Autosar element that should be created.
Example :	
createAutosarElement(EcuInstance.class);	

- boolean registerFile(String filename,String classname);

Description :	
Registers the given file to a class specified.	
Return Type :	
boolean	Whether the file is registered successfully or not

Parameters :	
filename	Name of the file that needs be registered
classname	Class type against which the files has to be registered
Example :	
registerFile("newFile.arxml ","TDATA-ARA");	

- String getAutosarReleaseVersion();

Description :	
Gives the autosar major and minor version of the project	
Return Type :	
String	Autosar version of the project
Example :	
getAutosarReleaseVersion();	

- ARPackage getLeafArPackage(final ARPackage parentArPackage)

Description :	
It gives the leaf level ARPackage in the hierarchy. Note: multiple child ARPackage would not work and will return the first available one.	
Return Type :	
ARPackage	Returns the child ARpackages of the parent package
Parameters :	
parentArPackage	parent Arpackage
Example :	
getLeafArPackage (parent package);	

- ARPackage createArPackage(final String arPackageName);

Description :	
Create ARPackages for the given string and string should be / seperated	
Return Type :	
ARPackage	Returns the root ARpackage
Parameters :	
arPackageName	ARPackages list separated by /
Example :	
createArPackage (“ArPackage1/ArPackage2”);	

- void addArPackagesToAutosarFileRoot(IFile autosarFile, final Object arPackageObject)

Description :

It adds the given arpackage to the autosarFile root	
Return Type :	
void	
Parameters :	
autosarFile	IFile from the Project project.get("relative-FilePath") would give the IFile
arPackageObject	Root ArPackage
Example :	
addArPackagesToAutosarFileRoot (file,rootPackage);	

- IFile createArxmlFile(String relativeFilePath);

Description :	
Creates the arxml file with AUTOSAR root. To create folder along with file then specify folder name followed by file name.	
Return Type :	
IFile	Returns the created Arxml IFile
Parameters :	
relativeFilePath	File name relative to project
Example :	
CreateArxmlFile ("myFile.arxml");	

- boolean deRegisterFile(String relativeFilePath);

Description :	
Deregister the file from the current project.	
Return Type :	
boolean	Whether the file successfully deregistered or not.
Parameters :	
relativeFilePath	Relative path of the file needs to be dereg-istered in project.
Example :	
deRegisterFile ("myFile.arxml");	

2.2.6 Variation Point Utilities

Variation Point Multiple Copy
public VariationPoint copy(VariationPoint vp[],OperatorEnum operator);

Description
Combines the two variation points, such that the result evaluates to true if one of the arguments evaluates to true. The result has the later binding time of the

two arguments, and condition is combined using the logical or operator ||. In case at least one of the two arguments is null, null shall be returned. In case both arguments have an identical condition, this a variation point with that condition shall be returned instead of the above described disjunction.

Return Type
Variation Point - Return the resulting variation point, or null if the array is empty

Parameters
- Vp[] - the array of variation points to be combined
- operator - Operator which specifies AND / OR variation point

Example
ISOLARLibraryUtils libraryUtil = inputContext.getModel(ISOLARLibraryUtils.class);
IVariationPointUtilities variationPointUtil = libraryUtil.getVariationPointUtilities();
VariationPoint vpOR = variationPointUtil.copy(variationPoint[], "&&");

Variation Point Deep Copy

VariationPoint copy(VariationPoint vp)

Description
Creates a deep copy of a variation point

Return Type
VariationPoint - returns the deep copy of vp

Parameters
- vp - the variation point to be copied

Example
ISOLARLibraryUtils libraryUtil = inputContext.getModel(ISOLARLibraryUtils.class);
IVariationPointUtilities variationPointUtil = libraryUtil.getVariationPointUtilities();
VariationPoint vpCopy = variationPointUtil.copy(<VP_INS>);

Evaluate Variation Point

EObject object= isActiveObject(EObject componentprototype, String binding-Time)

Description
Evaluate variation point of the given EObject

Return Type
String - Returns one of the following State UNRESOLVE, RESOLVED_ACTIVE, RE-SOLVED_INACTIVE

Parameters
- EObject - Element on which the Variation point needs to be evaluated
- bindingTime - [Optional] Evaluates the Variation point for given Binding Time. If not provide 'systemDesignTime' will be considered as default

Example
ISOLARLibraryUtils libraryUtil = inputContext.getModel(ISOLARLibraryUtils.class);
IVariationPointUtilities variationPointUtil = libraryUtil.getVariationPointUtili-ties();
boolean status = variationPointUtil.isActiveObject(componentprototype, "SYS-TEM-DESIGN-TIME");

2.2.7 Validation Utilities

AR Validation

boolean performValidation(String profile, List<EObject> scope, String version, String logFile, ILog log);

Description
This action helps the user to run the required validations on the selected pro-ject.

Return Type
Boolean - Returns true if the execution was successful else false

Parameters
- profile - [Optional] Validation Profile(System, Software, Ecu or BSW)
- scope - [Optional] The list of ArElements on which the validation to be

performed. For element specific validation scope value needs to be passed.
• Version - Autosar release version(4x or specific version could be mentioned)
• logFile - [Optional] command to log results in the specified file.(both relative and absolute path are supported)
• log - Console logger available in input context

Example

ISOLARLibraryUtils libraryUtil = inputContext.getModel(ISOLARLibraryUtils.class);
IValidationUtilities validationUtil = libraryUtil.getValidationUtilities();
boolean status = validationUtil.performValidation ("system",ecuList, "4x","log.txt", consoleLogger);

2.2.8 System Utilities

RTTM Import
boolean importRTTM(String ecucvalue, String mappingType, String input, String logFile, String additionalFile, ILog log);

Description
This action helps to auto generate the Runnable To Task mapping Containers from preconfigured excel file.

Return Type
Boolean - Returns true if the execution was successful else false

Parameters
• ecucvalue - Short name of the ecucValueCollection which contains Rte and Os configurations
• mappingType - Type of mapping (Entity2Task, Event2Task)
• input - Excel file path where Runnable| event to task mapping are configured
• logFile - Console logger available in input context
• additonalOptionalFile - [Optional] Additional preference input file path which contains the optional parameters which could be needed during execution. (both relative and absolute path is supported)
• log - Console logger available in input context

Example

ISOLARLibraryUtils libraryUtil = inputContext.getModel(ISOLARLibraryUtils.class);
ISystemUtilities systemUtil = libraryUtil.getSytemUtilities();

boolean status = systemUtil.importRTTM ("WiperControl","Event2Task","test-DataXcel.xlxs", "log.txt", "RTTMImport_CLI_Input.txt", consoleLogger);

Auto Signal Mapping

boolean autoSignalMapping(System system, String mappingType, String additionalFile, ILog log)

Description
Maps the Signal/SignalGroup to a DataElement in the system

Return Type
Boolean - Returns true if the execution was successful else false

Parameters
• system* - System instance in which mappings needs to be generated and related RootSwComposition will be considered for generating mappings
• additionalFile - [Optional] Additional preference input file path which contains the optional parameters which could be needed during execution. (both relative and absolute path is supported)
• mappingType - Mapping type that needs to be generated. o Possible types: SR - SenderReceiverToSignalMapping TR - TriggerToSignalMapping CD - SenderReceiverCompositeDataToSignalMapping CS - ClientServerToSignalMapping
• Log - Console logger available in input context

Example
ISOLARLibraryUtils libraryUtil = inputContext.getModel(ISOLARLibraryUtils.class);
ISystemUtilities systemUtil = libraryUtil.getSytemUtilities();
boolean status = systemUtil.autoSignalMapping(system,"SR","AutoSignalMapping_CLI_Input.txt" , consoleLogger);

Create EcuExtract

boolean createEcuExtract(EcuInstance ecu, String additionalFile, ILog log);

Description
This action is the extraction of ECU specific information from the System Configuration Description and also transforms the hierarchical System to a Flat view

Return Type
Boolean - Returns true if the execution was successful else false

Parameters
- ecuInstance - ECU Instance on which the ECU Extract is executed
- additionalFile - [Optional] Additional preference input file path which contains the optional parameters which could be needed during execution. (both relative and absolute path is supported)
- log - Console logger available in input context

Example
ISOLARLibraryUtils libraryUtil = inputContext.getModel(ISOLARLibraryUtils.class);
ISystemUtilities systemUtil = libraryUtil.getSytemUtilities();
boolean status = systemUtil.createEcuExtract(ecuInstance, "EcuExtract_CLI_Input.txt" , consoleLogger);

ASW Auto Generation

boolean aswAutoGen(List<EcuInstance> ecuList, String logFile, String additonalOptionalFile, ILog consoleLog)

Description
This action helps to generate ASW configuration from Communication Matrix information with a single click The below use cases are supported with this feature. - Generating ASW for the first time for a Communication Matrix - Updating the newly added communication matrix information into an existing ASW configuration

Return Type
Boolean - Returns true if the execution was successful else false

Parameters
- ecuList - List of Ecu's from which the ASW configuration will be generated.
- logFile : Optional command to log results in the specified file.
- additonalOptionalFile - [Optional] Additional preference input file path which contains the optional parameters which could be needed during execution. (both relative and absolute path is supported)

- log - Console logger available in input context

Example

ISOLARLibraryUtils libraryUtil = inputContext.getModel(ISOLARLibraryUtils.class);
ISystemUtilities systemUtil = libraryUtil.getSytemUtilities();
boolean status = systemUtil.aswAutoGen(ecuInstances, "log.txt", "AswAutoGeneration_CLI_Input.txt" , consoleLogger);

Perform Auto Cleanup

List performAutoCleanUp(System system, String logFile, ILog consoleLog)

Description

Removes the erroneous mapping from the system

Return Type

List - Returns a list of deleted erroneous mappings under the System ARElement

Parameters

- system - system instance where the erroneous mapping will be generated
- logFile : Optional command to log results in the specified file.
- additonalOptionalFile - [Optional] Additional preference input file path which contains the optional parameters which could be needed during execution. (both relative and absolute path is supported)
- log - Console logger available in input context

Example

ISOLARLibraryUtils libraryUtil = inputContext.getModel(ISOLARLibraryUtils.class);
ISystemUtilities systemUtil = libraryUtil.getSytemUtilities();
boolean status = systemUtil.performAutoCleanUp(system, "autoCleanUp.log" ,"AutoCleanUp_CLI_Input.txt" , log);

Generate System Extract

boolean generateSystemExtract(final List<EcuInstance> ecuInstances, final String system, final String targetProject, String additonalOptionalFile, final ILog consoleLog)

Description
This action helps to generates SystemExtract configuration for the selected EcuInstances & System. System Extract is the basis for designing subsystems. System Extract is not fully decomposed and still needs to be refined before it forms basis for ECU configuration

Return Type
Boolean - Returns true if the execution was successful else false

Parameters
- ecuInstances * - List of all EcuInstances (separated by comma) that needs to be extracted
- System -System instance from which the configuration needs to be extracted
- targetProject - Target project where SystemExtract needs to be generated. Could also specify absolute path if extract has to be generated outside the work-space
- additonalOptionalFile - [Optional] Additional preference input file path which contains the optional parameters which could be needed during execution. (both relative and absolute path is supported)
- log - Console logger available in input context

Example
ISOLARLibraryUtils libraryUtil = inputContext.getModel(ISOLARLibraryUtils.class);
ISystemUtilities systemUtil = libraryUtil.getSytemUtilities();
boolean status = systemUtil.generateSystemExtract(ecuInstances, "WiperCon-trol_System","SystemExtract" , "SystemExtract_CLI_Input.txt" , log);

Auto SWC To ECU Mapping

boolean autoSwcToEcuMapping(System system, CompositionSwCompo-nentType composition, EcuInstance ecuInstance, String additionalFile, String log-File, ILog log)

Description
Auto configures a composition by instantiating components, generating possible connections, mapping components to ECUs, deleting erroneous connections and generating ECUExtract

Return Type
Boolean - Returns true if the execution was successful else false

Parameters
- system - system instance under which the sw component prototypes shall be created
- composition* - Composition instance for which the swc to ecu mappings are generated
- ecuInstance - Ecu Instance for which the sw component prototypes will be created.
- additionalFile - [Optional] Additional preference input file path which contains the optional parameters which could be needed during execution. (both relative and absolute path is supported)
- logFile : Optional command to log results in the specified file.
- log - Console logger available in input context

Example
ISOLARLibraryUtils libraryUtil = inputContext.getModel(ISOLARLibraryUtils.class);
ISystemUtilities systemUtil = libraryUtil.getSytemUtilities();
boolean status = systemUtil.autoSwcToEcuMapping(system, composition, ecuInstance, "AutoSwcToEcuMapping_CLI_Input.txt", "logFile.txt", log);

Generate Cluster Extract

boolean createClusterExtract (final List<> clusters, final System system, String logFile, String additonalOptionalFile, final ILog consoleLog)

Description
This actions helps in extraction of CP Software Cluster specific information from the System Configuration Description

Return Type
Boolean - Returns true if the execution was successful else false

Parameters
- clusters - [Optional] list of cluster instances that needs to be extracted.If no clusters are provided,it will take all possible clusters from the system.
- System - [Optional] System instance from which cluster needs to be extracted.if system is not provided it will consider all system from the project.

- logFile : Optional command to log results in the specified file.
- additonalOptionalFile - [Optional] Additional preference input file path which contains the optional parameters which could be needed during execution. (both relative and absolute path is supported)
- log - Console logger available in input context

Example

ISOLARLibraryUtils libraryUtil = inputContext.getModel(ISOLARLibraryUtils.class);
ISystemUtilities systemUtil = libraryUtil.getSytemUtilities();
boolean status = systemUtil.createClusterExtract (clusters, system, "log.txt", "ClusterExtract_CLI_Input.txt" , this.log);

Auto SWC To Impl Mapping

boolean autoSwcToImplMapping(System system, boolean doCreateNewMappings, ILog consoleLog)

Description

This action helps to Auto generate the SwComponent To Implementation mapping

Return Type

Boolean - Returns true if the execution was successful else false

Parameters

- system - System instance where the SwcToImpl mappings will be generated
- doCreateNewMappings - Creates new SwcToImpl Mappings. If it is not specified, the existing mappings will be overridden
- Log - Console logger available in input context

Example

ISOLARLibraryUtils libraryUtil = inputContext.getModel(ISOLARLibraryUtils.class);
ISystemUtilities systemUtil = libraryUtil.getSytemUtilities();
boolean status = systemUtil.autoSwcToImplMapping(system, true, this.log);

2.2.9 Miscellaneous Utilities

Report generation

boolean reportGeneration(String reportID, String outputFilepath, ILog consoleLog);

Description
This action helps to generate excel reports of different ISOLAR Usecases Such as Datatypes, Component , Composition and EcuExtract informations

Return Type
Boolean - Returns true if the execution was successful else false

Parameters
• reportId - Each report id has different templates i.e possible report id's: o 1 : its generates DataTypes information o 2 : its generates Composition information o 3 : its generates EcuExtract information o 4 : its generates Component information o 8 : it generates report for ARXML Diff Merge in the excel formato -1 : its generates complete information
• outputFilePath - File path where the report needs to be generated(default path is C:\Temp\GenReport.xls)
• log - Console logger available in input context
• inputFiles:[Conditionally Optional] Comma separated arxml absolute file paths. It is mandatory when the --reportid given with the value "8"

Example
ISOLARLibraryUtils libraryUtil = inputContext.getModel(ISOLARLibraryUtils.class);
IMiscellaneousUtilities miscellaneousUtil = libraryUtil.getMiscellaneousUtilities();
boolean status = miscellaneousUtil.reportGeneration("1","c:\temp\", consoleLogger);

Create AnyInstanceRef from ComponentPrototypeRef

AnyInstanceRef asAnyInstanceRef (ComponentInSystemInstanceRef comp)

Description
Converts a reference to a prototype of a software component to an object of type AnyInstanceRef. This is required e.g. in the ECUC values for ECUC partitions, see EcucPartitionSoftwareComponentInstanceRef.

Return Type

AnyInstanceRef - an object of type AnyInstanceRef that refers to the argument

Parameters

- comp - the component prototype instance refernence

Example

ISOLARLibraryUtils libraryUtil = inputContext.getModel(ISOLARLibraryUtils.class);
IMiscellaneousUtilities miscellaneousUtil = libraryUtil.getMiscellaneousUtilities();
AnyInstanceRef anyInstance = miscellaneousUtil.asAnyInstanceRef(<CPT_INSTANCE_REF>);

Create ComponentPrototypeInstance from AnyInstanceRef

ComponentInSystemInstanceRef asComponentInSystemInstanceRef
(AnyInstanceRef ref)

Description

Converts an objects of type AnyReference to a reference to a software component prototype

Return Type

ComponentInSystemInstanceRef - An object of type ComponentInSystemInstanceRef that refers to the argument

Parameters

- ref - AnyInstanceRef from which the ComponentSystemIsntanceRef will be created

Example

ISOLARLibraryUtils libraryUtil = inputContext.getModel(ISOLARLibraryUtils.class);
IMiscellaneousUtilities miscellaneousUtil = libraryUtil.getMiscellaneousUtilities();
ComponentInSystemInstanceRef componentInSystemInstanceRef = miscellaneousUtil.asComponentInSystemInstanceRef(<ANY_INSTANCE_REF>);

Refers to Same Instance Ref

boolean refersToSameInstance(AtpInstanceRef a, AtpInstanceRef b);

Description
Checks if two instance references refer to the same object.

Return Type
boolean - Returns true if the execution was successful else false

Parameters
- AtpInstanceRef - AtpInstanceRef object a
- AtpInstanceRef - AtpInstanceRef object b

Example
ISOLARLibraryUtils libraryUtil = inputContext.getModel(ISOLARLibraryUtils.class);
IMiscellaneousUtilities miscellaneousUtil = libraryUtil.getMiscellaneousUtilities();
boolean status = miscellaneousUtil.refersToSameInstance(<AtpInstanceRef>,<AtpInstanceRef>);

2.2.10 Composition Utilities

Auto Connection

public void autoConnect(String connectionType, CompositionSwComponentType composition, String mappingFile, String targetFile, ILog consoleLog);

Description
Auto-generates the connections within a given composition

Return Type
Boolean - Returns true if the execution was successful else false

Parameters
- connectionType : Type of the connection(assembly,delegation,passthrough)
- composition* - CompositionSwComponentType on which the autogeneration is done
- targetFile - the file where the new connectors should be stored. Project-local workspace path
- additonalOptionalFile : [Optional] Additional preference input file path. (both

relative and absolute path is supported)
• log - Console logger available in input context

Example
ISOLARLibraryUtils libraryUtil = inputContext.getModel(ISOLARLibraryUtils.class);
ICompositionUtilities compositionUtil = libraryUtil.getCompositionUtilities();
compositionUtil.autoConnect("Assembly", composition, "autoConnect.arxml", "autoconnect_cli_input.txt", log);

Delete Erroneous Connection

boolean deleteErroneousConnection(CompositionSwComponentType composition,String logFile, String additionalFile , ILog log);

Description
This action shall invoke Delete Erroneous Connections feature using which all invalid connectors in a Composition can be deleted

Return Type
Boolean - Returns true if the execution was successful else false

Parameters
• composition* - CompositionSwComponentType element for which the erroneous connections shall be deleted
• logFile : Optional command to log results in the specified file.
• additionalFile - [Optional] Additional preference input file path which contains the optional parameters which could be needed during execution. (both relative and absolute path is supported)
• log - Console logger available in input context

Example
ISOLARLibraryUtils libraryUtil = inputContext.getModel(ISOLARLibraryUtils.class);
ICompositionUtilities compositionUtil = libraryUtil.getCompositionUtilities();
compositionUtil.deleteErroneousConnection(composition, "log.txt", "DeleteConnection_CLI_input.txt" , consoleLogger);

Adapter Generation

boolean generateAdapterGeneration(CompositionSwComponentType compIns, String additionalFile , ILog log);

Description
This action shall invoke Adapter Generation feature on a Composition. Adapter Generation generates stub Components which shall be used to test developed Components by successful generation of RTE

Return Type
Boolean - Returns true if adapters are generated successfully else false

Parameters
- compIns* - Composition instance on which the Adapter Generation is invoked.
- additionalFile - [Optional] Additional preference input file path which contains the optional parameters which could be needed during execution. (both relative and absolute path is supported)
- Log - Console logger available in input context

Example
ISOLARLibraryUtils libraryUtil = inputContext.getModel(ISOLARLibraryUtils.class);
ICompositionUtilities compositionUtil = libraryUtil.getCompositionUtilities();
compositionUtil.generateAdapterGeneration(composition, "AdapterGen_CLI_Input.txt" , consoleLogger);

Auto Configure Composition
boolean autoConfigureComposition(CompositionSwComponentType swComposition, String additonalOptionalFile, ILog consoleLog);

Description
Auto configures a composition by instantiating components, generating possible connections, mapping components to ECUs, deleting erroneous connections and generating ECUExtract

Return Type
Boolean - Returns true if the execution was successful else false

Parameters
- swComposition* - CompositionSwComponentType on which the

Autoconfiguration is done
• additonalOptionalFile - [Optional] Additional preference input file path which contains the optional parameters which could be needed during execution. (both relative and absolute path is supported)
• Log - Console logger available in input context

Example

ISOLARLibraryUtils libraryUtil = inputContext.getModel(ISOLARLibraryUtils.class);
ICompositionUtilities compositionUtil = libraryUtil.getCompositionUtilities();
boolean status = compositionUtil.autoConfigureComposition(<Composition_Instance>, "c:/temp/preferenceFile.txt", consoleLogger);

2.2.11 Component Utilities

Generate Code Frame

boolean generateCodeFrame(String component, String destinationFolder , String additionalFile, ILog log);

Description

This action helps to generation of C files for the software components once the user has completed their configuration. The generated C file for a component is a template containing Runnable methods and RTE API calls based on the configuration of the RunnableEntity within the Component. All the RunnableEntities' corresponding C methods can be grouped within a single C file template or a separate C file can be generated for each of the different RunnableEntities within a component.

Return Type

Boolean - Returns true if the execution was successful else false

Parameters

• component - Absolute Qualified name of the component.
• destinationFolder folder - Destination folder where code frame files will be created.
• additionalFile - [Optional] Additional preference input file path which contains the optional parameters which could be needed during execution. (both relative and absolute path is supported)
• log - Console logger available in input context

Example

ISOLARLibraryUtils libraryUtil = inputContext.getModel(ISOLARLibraryUtils.class);
IComponentUtilities componentUtil = libraryUtil.getComponentUtilities();

boolean status = componentUtil.generateCodeFrame(swComponent, "c:/temp" "GenerateCodeFrame_CLI_Input.txt", consoleLogger);

Adapter Generation
boolean generateAdapterGeneration(SwComponentType compIns, String additionalFile , ILog log);

Description
This action shall invoke Adapter Generation feature on a Component. Adapter Generation generates stub Components which shall be used to test developed Components by successful generation of RTE

Return Type
Boolean - Returns true if adapters are generated successfully else false

Parameters
- compIns* - Component instance on which the Adapter Generation is invoked.
- additionalFile - [Optional] Additional preference input file path which contains the optional parameters which could be needed during execution. (both relative and absolute path is supported)
- log - Console logger available in input context

Example
ISOLARLibraryUtils libraryUtil = inputContext.getModel(ISOLARLibraryUtils.class);
IComponentUtilities componentUtil = libraryUtil.getComponentUtilities();
compositionUtil.deleteErroneousConnection(swComponent, "AdapterGen_CLI_Input.txt", consoleLogger);

2.2.12 Platform Utilities

Get AR Elements

List<T> getElements(final Class<T> type)

Description
This action provides all the merged elements of the given class type specified from the project

Return Type
List<T> - Returns the list merged element

Parameters

- type - Name of the inputElement class

Example

ISOLARLibraryUtils libraryUtil = inputContext.getModel(ISOLARLibraryUtils.class);
IPlatformUtilities platformUtil = libraryUtil.getPlatformUtilities();
List<T> list = platformUtil.getElements(EcuInstance.class);

Register Arxml

boolean registerFile(String filename,String classname);

Description

This action helps to register the given file to a class specified.

Return Type

Boolean - Returns true if the execution was successful else false

Parameters

- fileName - Name of the file that needs be registered
- className - Class type against which the files has to be registered

Example

ISOLARLibraryUtils libraryUtil = inputContext.getModel(ISOLARLibraryUtils.class);
IPlatformUtilities platformUtil = libraryUtil.getPlatformUtilities();
boolean status = platformUtil.registerFile("newFile.arxml","TDATA-ARA");

Create Autosar Element

EObject createAutosarElement(Class classType, String fqn);

Description

This action helps to create Autosar element for the specified class type.

Return Type

EObject - Returns the created Autosar element

Parameters

- class - Class type Autosar element that should be created.
- fqn - FullyQuaified path of new Created EObject

- file - IFile relative path

Example

ISOLARLibraryUtils libraryUtil = inputContext.getModel(ISOLARLibraryUtils.class);
IPlatformUtilities platformUtil = libraryUtil.getPlatformUtilities();
EObject object = platformUtil.createAutosarElement(EcuInstance.class, "/WiperControl-CommMatrix/Ecu_PackageSw/WiperControlEcu", "newFile.arxml");

Create Autosar Element

EObject createAutosarElement(Class classType, String fqn);

Description

This action helps to create Autosar element for the specified class type.

Return Type

EObject - Returns the created Autosar element

Parameters

- class - Class type Autosar element that should be created.

Example

ISOLARLibraryUtils libraryUtil = inputContext.getModel(ISOLARLibraryUtils.class);
IPlatformUtilities platformUtil = libraryUtil.getPlatformUtilities();
EObject object = platformUtil.createAutosarElement(EcuInstance.class);

Get Autosar release Version

String getAutosarReleaseVersion()

Description

This action helps to give the autosar major and minor version of the project

Return Type

String - Autosar version of the project

Example

ISOLARLibraryUtils libraryUtil = inputContext.getModel(ISOLARLibraryUtils.class);
IPlatformUtilities platformUtil = libraryUtil.getPlatformUtilities();

String version = platformUtil.getAutosarReleaseVersion();

Get Unmerged Elements with Qualified path
<T> List<T> getUnmergedElements(String fqn, Class<T> type)

Description
It gives the unmerged instance of the class type specified with particular FQN.

Return Type
Returns the list of actual instance of the EObject

Parameters
- fqn : Absolute Qualified path of the element to be returned
- type :Class type of the element that needs to be fetched and returned

Example
ISOLARLibraryUtils libraryUtil = inputContext.getModel(ISOLARLibraryUtils.class);
IPlatformUtilities platformUtil = libraryUtil.getPlatformUtilities();
List<T> list = platformUtil.getUnmergedElements("/WiperControlCommunicationMa-trix/Pkg_Ecu/WiperControl","EcuInstance.class");

Get Unmerged Elements
<T extends EObject> List<T> getUnmergedElements(Class<T> type)

Description
It gives the unmerged elements of the class type specified from the project on which java action is executed.

Return Type
Class type of which elements needs to be fetched and returned

Parameters
- type : Class type of the element that needs to be fetched and returned

Example
ISOLARLibraryUtils libraryUtil = inputContext.getModel(ISOLARLibraryUtils.class);
IPlatformUtilities platformUtil = libraryUtil.getPlatformUtilities();
List<EObject> obj = platformUtil.getUnmergedElements(EcuInstance.class);

Add ARPackage To Root
void addArPackagesToAutosarFileRoot(IFile autosarFile, final Object arPackageObject)

Description
This action helps to add the given arpackage to the autosarFile root

Return Type

Parameters
- autosarFile - IFile from the Project project.get("relativeFilePath") would give the IFile
- arPackagePath - Root ArPackage

Example
ISOLARLibraryUtils libraryUtil = inputContext.getModel(ISOLARLibraryUtils.class);
IPlatformUtilities platformUtil = libraryUtil.getPlatformUtilities();
platformUtil.addArPackagesToAutosarFileRoot(file, rootPackage);

Update Model
<O, I> O updateModel(Function<I, O> function, I object)

Description
Any model update that needs to be performed should be included in a model update custom class (with method public IStatus method(CustomClass object)) and the object needs to be passed in the updateModel API.

Return Type
Returns the function

Parameters
- function : Name of the file to be parsed.
- object :Separator which separates the key and values

Example
ISOLARLibraryUtils libraryUtil = inputContext.getModel(ISOLARLibraryUtils.class);
IPlatformUtilities platformUtil = libraryUtil.getPlatformUtilities();
Object obj = platformUtil.updateModel();

Get All Resources
List<Resource> getAllResources()

Description
It returns all the autosar resources files in the given project

Return Type
List<Resource> - returns list all the autosar resources within given project

Example
ISOLARLibraryUtils libraryUtil = inputContext.getModel(ISOLARLibraryUtils.class);
IPlatformUtilities platformUtil = libraryUtil.getPlatformUtilities();
List<Resource> resourceList = platformUtil.getAllResources();

Get AR Element using FQN
<T extends EObject> T getElement(final String fqn, final Class<T> type)

Description
This action provides the merged instance of the class type specified with particular FQN.

Return Type
Returns the merged instance the EObject

Parameters
- fqn- Absolute Qualified path of the element to be returned
- type - Class type of the element that needs to be fetched and returned

Example
ISOLARLibraryUtils libraryUtil = inputContext.getModel(ISOLARLibraryUtils.class);
IPlatformUtilities platformUtil = libraryUtil.getPlatformUtilities();
EcuInstance ecu = platformUtil.getElement("/WiperControlCommunicationMa-trix/Pkg_Ecu/WiperControl", EcuInstance.class);

Create Arxml
IFile createArxmlFile(String relativeFilePath)

Description
Creates the arxml file with AUTOSAR root. To create folder along with file then specify folder name followed by file name.

Return Type
Returns the created Arxml IFile

Parameters
- relativeFilePath : File name relative to project

Example
ISOLARLibraryUtils libraryUtil = inputContext.getModel(ISOLARLibraryUtils.class);
IPlatformUtilities platformUtil = libraryUtil.getPlatformUtilities();
IFile file = platformUtil.CreateArxmlFile ("myFile.arxml");

Get Input Element with Split layer
List<Object> getInputElement()

Description
Selected element in UI or CLI will be return by this API in java file. This API is deprecated, use @getInputElement() API to fetch the selected element.

Return Type
Returns selected inputElement

Parameters
- isUseSplitLayer : Passing true is same as invoking getInputElement(), passing false would return the actual element selected input element

Example
ISOLARLibraryUtils libraryUtil = inputContext.getModel(ISOLARLibraryUtils.class);
IPlatformUtilities platformUtil = libraryUtil.getPlatformUtilities();
List<Object> list = platformUtil.getInputElement(false);

Get Input Element
List<Object> getInputElement()

Description
Selected element in UI or CLI will be return by this API in java file. This will merge all the split elements and return as a merged element

Return Type
Returns the inputElement

Example

```
ISOLARLibraryUtils libraryUtil = inputContext.getModel(ISOLARLibraryUtils.class);
IPlatformUtilities platformUtil = libraryUtil.getPlatformUtilities();
List<Object> list = platformUtil.getInputElement();
```

Parse Property file

Map<String, String> parsePropertyFile(String relativeFilePath, String keyValueSeperator)

Description
It parses the property file given and returns the map with key and value.

Return Type
Returns the Map of key value pairs

Parameters
- relativeFilePath : Name of the file to be parsed.
- Separator which separates the key and values

Example

```
ISOLARLibraryUtils libraryUtil = inputContext.getModel(ISOLARLibraryUtils.class);
IPlatformUtilities platformUtil = libraryUtil.getPlatformUtilities();
Map<String, String> map = platformUtil.parsePropertyFile("pasefile.txt", "=");
```

Parse property file with value seperator
Map<String, List<String>> parsePropertyFile(String relativeFilePath, String keyValueSeperator, String valueSeperator)

Description
It parses the property file given and returns the map.

Return Type
Returns the Map of key & list of values associated with the key.

Parameters
- relativeFilePath : Name of the file to be parsed.

- keyValueSeparator : Separator which separates the key and values
- valueSeparator : Separator which separates the list value.

Example

ISOLARLibraryUtils libraryUtil = inputContext.getModel(ISOLARLibraryUtils.class);
IPlatformUtilities platformUtil = libraryUtil.getPlatformUtilities();
Map<String, List<String>> map = platformUtil.parsePropertyFile("parsefile.txt", "=", ",")

Get Leaf ARPackage

ARPackage getLeafArPackage(final ARPackage parentArPackage)

Description

This action helps to provide the leaf level ARPackage in the hierarchy. Note: multiple child ARPackage would not work and will return the first available one.

Return Type

ARPackage - Returns the root ARpackage

Parameters

- parentArPackage - parent Arpackage

Example

ISOLARLibraryUtils libraryUtil = inputContext.getModel(ISOLARLibraryUtils.class);
IPlatformUtilities platformUtil = libraryUtil.getPlatformUtilities();
ARPackage arPackage = platformUtil.getLeafArPackage(parentPackage);

Create ARPackage

ARPackage createArPackage(final String arPackageName)

Description

Create ARPackages for the given string and string should be / separated

Return Type

ARPackage - Returns the root ARpackage

Parameters

- arPackageName - AR Package name

Example

ISOLARLibraryUtils libraryUtil = inputContext.getModel(ISOLARLibraryUtils.class);

```
IPlatformUtilities platformUtil = libraryUtil.getPlatformUtilities();
ARPackage arPackage = platformUtil.createArPackage ("ArPackage1/ArPackage2");
```

3 Example Projects

3.1 Java Action Example Project

ISOLAR-A provides a Java Action examples project which can be imported into the workspace. This project contains example Java Action files for simple use cases like creating auto signal maping, create Ecu extract, update model etc. These Java Action files provide a glimpse of how Java Action feature is used in ISOLAR-A.

Script project can be imported from **New > ISOLAR-A > ISOLAR-A Vendor Extension Template Examples > JavaActionExamples**

Some of the Java Action files present in the project are –

1. ExecutingIsolarApplications.java – generates Auto Signal Mapping, Ecu extract,Auto connection etc
2. CreateApplicationSwComponent.java – sample java action which creates an ApplicationSwComponent and a PPortPrototype
3. GenerateCSVExample.java – Sample java action file to read and write CSV file.
4. ReadNdWriteExcelExample.java – Sample java action file to read and write Excel file
5. ComponentUpdateExample.java – Sample Java action file to update the component short name and provided service instance to the component.

After Enable Java Action, Menu options will appear on the input element for component and on project for others. Clicking on the option will run the Java Action. More details on how to configure Java headers for execution is available in the tool help **ISOLAR-A User Guide > Scripting and Productive Features > JAVA Action**.

- To execute through CLI mode, use command:

  - -runjava --project=C:\temp\javaActionExample --class= SampleJavaAction --input-type= autosar40.system.fibex.fibexcore.coretopology.EcuInstance --inputelements= WiperControl

- To execute through UI: use ISOLAR-A Java Action

3.2 Scripting Example Projects

ISOLAR-A provides a scripting examples project which can be imported into the workspace. This project contains example scripts for simple use cases like finding open ports for the Composition, iterating through the connectors in the Composition, AutoCleanup of System etc. These scripts provide a glimpse of how scripting use cases can be achieved in ISOLAR-A.

Script project can be imported from **New > ISOLAR-A > ISOLAR-A Vendor Extension Template Examples > ScriptingEnvironmentExamples**

Some of the Scripts present in the project are –

1. OpenPortsConnections.js – Finds open ports in a given Composition
2. FindAndDeleteErroneousConnections.js – Finds and deletes erroneous connections in a given Composition
3. ComponentDataDictionary.js – Returns a list of Data Dictionary elements used for a given Component
4. AutoCleanUpSwcMappings.js – Deletes erroneous mappings in a System

To see these scripts in action, add the location of the project to **Window > Preferences > Scripting > Script Locations**

Once the location is added, Menu options will appear for 1 and 2 for the Composition, 3 for Component and 4 for System element. Clicking on the option will run the script. More details on how to configure Script headers for execution is available in the tool help **ISOLAR-A User Guide > Scripting and Productive Features > ISOLAR-A Scripting**.

A script example that could be executed by bamf, CLI and UI is available under BAMFEnvironmentExamples.

New > ISOLAR-A > ISOLAR-A Vendor Extension Template Examples > BAMFEnvironmentExamples

- To execute through bamf use action: **ExecuteCommonScript**
- To execute through CLI mode, use command:

  runscript -p="<Absolute path of BAMFEnvironmentExamples>" -s="<Absolute path of SystemCreationScript.js present under BAMFEnvironmentExamples>" EXTR_WiperControl
- To execute through UI: use ISOLAR-A Scripting

4 Contact Information

Technical Support

For details of your local sales office as well as your local technical support team and product hotlines, take a look at the website: www.etas.com/hotlines

ETAS Headquarters

ETAS GmbH
Borsigstraße 24
70469 Stuttgart
Germany

Phone: +49 711 3423-0
Fax: +49 711 3423-2106
Internet: www.etas.com